

Tartalom:

1. BEVEZETÉS 1.1. A PROGRAM CÉLJA 1.2. KÖRNYEZET 2. FELÉPÍTÉS 2.1. FILEOK 2.2. FORDÍTÁSI & FUTTATÁSI KÖRNYEZET 3. ADATSZERKEZETEK 3.1. ALAP KONSTANSOK 3.2. FELELETVÁLASZTÓS KÉRDÉS – CHOOSEQUESTION 3.3. SORRENDEZŐS KÉRDÉS – ORDERQUESTION 3.4. DICSŐSÉGLISTA ELEM – HIGHSCORE 4. FŐ MŰKÖDÉSI FOLYAMAT 4.1. INDÍTÁS – MAIN.C 4.2. KÉRDÉSFÁJLOK BETÖLTÉSE 4.3. FŐMENÜ (MAIN CIKLUS) 4.4. Játékmenet – playGame()	5. FÁJLKEZELÉS ÉS FORMÁTUMOK 5.1. KÉRDÉSFÁJLOK 5.2. DICSŐSÉGLISTA – DICSOSEGLISTA.CSV 6. DINAMIKUS MEMÓRIAKEZELÉS 6.1. KÉRDÉSEK SZTRINGJEINEK TÁROLÁSA 6.2. DICSŐSÉGLISTA DINAMIKUS TÖMB 6.3. NYEREMÉNY FORMÁZÁSA – FORMATPRIZE() 7. SEGÉDFÜGGVÉNYEK & LOGIKA 7.1. KONZOL 7.2. BIZTONSÁGOS SZINTEK 7.3. SEGÍTSÉGEK 8. HIBAKEZELÉS, KORLÁTOK, FELTÉTELEZÉSEK 8.1. CSV HIBÁK 8.2. MEMÓRIAFOGLALÁSI HIBÁK 8.3. MAXIMÁLIS ELEMESZÁMOK 9. *TOVÁBBFEJLESZTÉSI ÖTLETEK
--	---

1. Bevezetés

1.1. A program célja

- A „Legyen Ön is Milliomos” egy konzolos kvízzjáték, ami a TV-s műsor szabályait követi
- A játékosnak egymás után 12 egyre nehezebb kérdésre kell válaszolnia
- Minél tovább jut, annál több pénzt gyűjt össze, rossz válasz esetén viszont visszaesik a legutóbb elért biztos szintre

1.2. Környezet

- A program Windows konzolon fut, C nyelven készült
- Fordítás: `gcc.exe -Wall -Wextra -g3 game.c main.c -o loim.exe`

2. Felépítés

2.1. Fileok

2.1.1. main.c – Főprogram

- Itt van a `main()` belépési pont
- Itt állítjuk be a konzolt, töltjük be a kérdésfájlokat, és kezeljük a főmenüt
- A játék logikája a `playGame()` függvényben van

2.1.2. game.c – Segédfüggvények

- Kérdésbetöltő / felszabadító függvények.
- Dicsőséglista írása és olvasása, rendezése, formázott kiírás
- Logikai segédfüggvények (safe szintek, segítségék)

2.1.3. game.h – Fejlécfile

- Konstansok (szintek száma, maximális kérdésszámok, játékmódok)
- Adatszerkezetek deklarációi (ChooseQuestion, OrderQuestion, HighScore)
- Függvény fejlécek definiálása

2.2. Fordítási & Futtatási környezet

- Windows specifikus részek: `windows.h`, `SetConsoleOutputCP`, `SetConsoleCP` – ezekkel lesz a konzol UTF-8
- A `debugmalloc.h` eltávolítása ha nem akarjuk minden futás után megjeleníteni a diagnosztikát

3. Adatszerkezetek

3.1. Alap konstansok

3.1.1. Szint & Max Kérdésszám

- `#define LEVELS 12` – Összesen 12 szint van
- `#define MAX_CHOOSE_QUESTIONS_PER_LEVEL 600` – Ennyi feleletválasztós lehet max egy szinten
- `#define MAX_ORDER_QUESTIONS 200` – Max 200 sorrendező kérdés

3.1.2. Szöveg & Max Név hosszak

- `MAX_TXT 256` – Kérdések / válaszok max hossza
- `MAX_NAME 21` – Játékosnév max hossza (20 karakter + lezáró \0)

3.1.3. Pénzlétra

- `static const int PRIZE_LADDER[LEVELS] = { ... }` – A 12 lépcsős pénzüsszegek (5000 -> 5 000 000)

3.1.4. Játékmódok

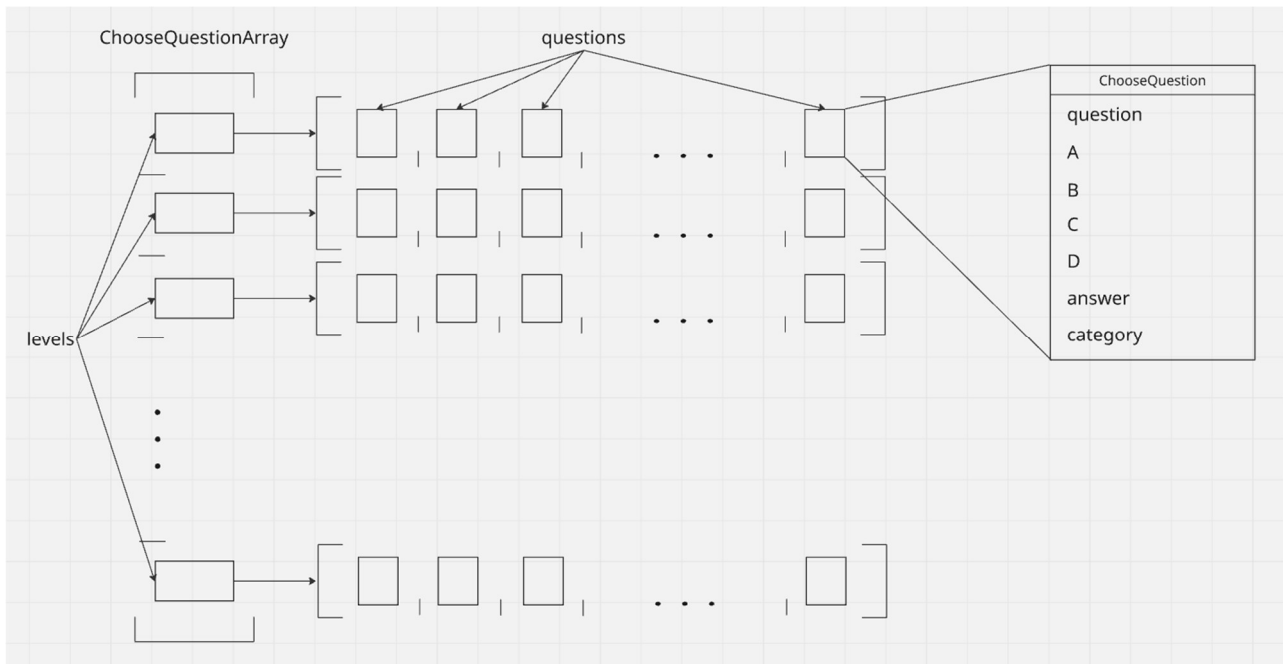
- `MODE_CHOOSE` – Csak feleletválasztós
- `MODE_ORDER` – Csak sorrendező
- `MODE_BOTH` – Vegyes mód (minden 3. szint sorrendező)

3.2. Feleletválasztós kérdés – ChooseQuestion

- `int diff` – nehézségi szint (1 -> LEVELS)
- `char *q` – kérdés szövege (dinamikusan foglalva)
- `char *answers[4]` – A, B, C, D válaszlehetőségek
- `char *correct` – helyes válasz betűjele (pld: "A")
- `char *cat` – kategória (pld: „Történelem”)

3.2.1. Kérdések tárolása szintek szerint

- `typedef ChooseQuestion ChooseQuestionArray[LEVELS][MAX_CHOOSE_QUESTIONS_PER_LEVEL];`
 - Két dimenziós tömb (szint x kérdések -> bennük struct-ok)



3.3. Sorrendező kérdés – OrderQuestion

- `char *q` – sorkezdés szövege
- `char *answers[4]` – 4 elem (A, B, C, D) amiket helyes sorrendbe kell rakni
- `char *correct` – helyes sorrend, pld: "ACBD"
- `char *cat` – kategória
- A kérdések egy sima egydimenziós tömbben vannak: `OrderQuestion questions[MAX_ORDER_QUESTIONS];`

3.4. Dicsőséglista elem – HighScore

- `char *name` – név
- `int prize` – nyeremény
- `char *date` – dátum (ÉÉÉÉ/HH/NN)
- Beolvasáskor dinamikusan növelt tömbben tároljuk.

4. Fő működési folyamat

4.1. Indítás – main.c

4.1.1. Konzol inicializálása

- `consoleUtf8()` – állítja be az UTF-8-at
- ékezetes betűk + színes ANSI működjenek (csak stílushoz kell)

4.1.2. Véletlenszám-generátor

- `srand((unsigned int)time(NULL));` – így nem ismétlődnek a kérdéssorrendek.

4.1.3. Debugmalloc beállítás

- `debugmalloc_max_block_size(32L * 1024 * 1024);` – max 32 MB-os blokkokat figyel
- Át kell állítani mert a kérdéseink az eredeti értéket meghaladják

4.1.4. Dinamikus adatszerkezetek

- `ChooseQuestionArray *cqdata = calloc(1, sizeof(ChooseQuestionArray));`

- A feleletválasztós kérdések nagy 2D tömbje calloc-kal jön létre.
- A sorrendező kérdések statikus tömbben vannak

4.2. Kérdésfájlok betöltése

4.2.1. Feleletválasztós kérdések – `loadChooseQuestions()`

- BE: fájlnev, két-dimenziós kérdéstömb pointere, cqCounts
- Megnyitja a CSV-t, kihagyja a fejlécsort
- Minden sorban:
 - PONTOSVESSZŐK számlálása soronként (7 db-ot vár) -> hibás sorok átugrása
 - `strtok(line, ";")` -> nehézségi szint
 - `atoi`-val konvertálás, 0 -> LEVELS-1 intervallum ellenőrzése
 - Szintenkénti limit (`MAX_CHOOSE_QUESTIONS_PER_LEVEL`) ellenőrzése
 - `Strdup` mindenre -> majd növeljük a darabszámot.
- KI: siker = 1, hiba/üres = 0

4.2.2. Sorrendező kérdések – `loadOrderQuestions()`

- BE: fájlnev, sorrendező kérdések tömbje, betöltött darabszám pointere
- Megnyitja a CSV-t, fejlécsor átugrása
- Minden sorban:
 - PONTOSVESSZŐK számlálása soronként (6 db-ot vár) -> hibás sorok átugrása
 - Ha jó a sor, akkor: q, 4 válasz, helyes sorrend, kategória - mind `strdup`-olva
 - `(*orderCount)++` növelése
- KI: siker = 1, hiba/üres = 0

4.2.3. Fileok léteznek? – `fileExists()`

- Ellenőrzi hogy a megadott file létezik-e. Ha nem akkor a program megáll

4.3. Főmenü (Main ciklus)

4.3.1. Menü elemek

- [1] ÚJ JÁTÉK
- [2] DICSŐSÉGLISTA
- [3] KILÉPÉS

4.3.2. Menü logika

- Beolvas egy karaktert, `atoi`-val számmá alakítja azt
- `switch(choice)` alapján:
 1. játékmód választása (ha nincs egy adott file, a játékosnak fel se ajánljuk)
 2. `showHighscores()` meghívása
 3. kilépés, memória felszabadítása (`freeChooseQuestions`, `freeOrderQuestions`)
 - Egyéb: hibaüzenet, menü újra kiírása

4.4. Játékmenet – `playGame()`

4.4.1. Játékos név bekérése – `getPlayerName()`

- max 20 karakter, üres input esetén „Jatekos”

4.4.2. Változók

- `currentLevel` – aktuális szint indexe (0 -> LEVELS-1)
- `gameOver`, `walkAway` – játék végét indikáló változók
- `finalPrize` – végső nyeremény
- `used5050`, `usedAudience` – segítségék már felhasználva?
- `hiddenResponse[2]` – 50:50 során elrejtett válaszindexek (0...3 vagy -1)

4.4.3. Kérdéstípus kiválasztása

- `isOrderLevel` – boolean, az adott szint sorrendező-e?
- Ha `MODE_ORDER`: minden szint sorrendező
- Ha `MODE_BOTH`: minden 3. kérdés sorrendező: $(currentLevel + 1) \% 3 == 0$
- Ha `MODE_CHOOSE`: minden kérdés feleletválasztós

4.4.4. Fejléc & Pénzlétra kiírása

- `formatPrize(PRIZE_LADDER[currentLevel])` -> „5.000.000” formájú string
- Kiírás: sorszámozott kérdés + az aktuális tét
- Ha létezik már biztos nyeremény (`getSafePrize(currentLevel - 1) > 0`) akkor azt is kiírja

4.4.5. Feleletválasztós kérdés menete

- Véletlen index: `rand() % cqCounts[currentLevel]`
- `cq = &(*cqdata)[currentLevel][r];` – pointer az adott kérdésre

- Kiírja a témát, kérdést, A–D válaszokat
- `correctAnsStr` -> helyes válasz egybetűs stringként (pld: „A”)

4.4.6. Sorrendezős kérdés menete

- Véletlen index: `rand() % oqCount`
- `oq = &oqdata[r];` – kiválasztott sorrendezős kérdés
- Kiírja a kategóriát, kérdést, A–D elemeket egymás alá
- `correctAnsStr` -> helyes sorrend ("ABCD"), toupper-ral normalizálva

4.4.7. Válasz bekérése és érvényesítés

- `fgets(input, sizeof(input), stdin)` + `strcspn`-nal „\n” levágása
- Minden karaktert nagybetűre alakít
- **Megállás:** "Q" -> kilépés a játékból, `finalPrize` = az előző szint pénze
- **Segítségek** (csak feleletválasztós kérdésnél):
 - "F" -> 50:50 (ha még nem használta)
 - "K" -> közönség segítsége (ha még nem használta)
- **Érvényes formátum:**
 - Sorrendezős: 4 karakter, mind A–D közötti
 - Feleletválasztós: 1 karakter, A–D, és nem tartozik az 50:50-el elrejtett válaszok közé (ha alkalmazta)

4.4.8. Kiértékelés & Szint léptetése

- Ha nem szállt ki (!walkAway):
 - Ha `strcmp(input, correctAnsStr) == 0`:
 - Helyes válasz -> zöld kiírás, safe szint elérése esetén külön üzenet
 - `currentLevel++` és `waitEnter() + system("cls")`
 - Egyébként:
 - Rossz válasz -> piros üzenet + helyes válasz kiírása
 - `finalPrize = getSafePrize(currentLevel - 1)`
 - Nyeremény kiírása, `gameOver = 1`

4.4.9. Játék vége & Nyeremény mentése

- Ha a játékos elérte az utolsó szintet (`currentLevel == LEVELS`)
 - „Ön milliomos lett!” és `finalPrize = PRIZE_LADDER[LEVELS - 1]`
- Ha a játékos kiszállt (`walkAway`) -> `finalPrize` az előző szint összege (vagy 0).
- Minden esetben:
 - `saveHighscore(playerName, finalPrize);`
 - `waitEnter();`

5. Fájlkezelés és formátumok

5.1. Kérdésfájlok

5.1.1. Feleletválasztós file – `kerdesek.csv`

- Fejlécsor utáni formátum: Nehézség;Kérdés;A;B;C;D; Helyes;Kategória
- A betöltő függvény:
 - Soronként beolvasás
 - Ellenőrzi a pontosvesszők számát (7) -> üres / hibás sorok átugrása
 - `strtok`-kal darabolás és `strdup`-pal tárolás

5.1.2. Sorrendezős file – `sorkerdesek.csv`

- Fejlécsor utáni formátum: Kérdés;A;B;C;D;Helyes;Kategória
- Ugyanaz az elv: pontosvessző számolás (6), `strtok`, `strdup`, hibás sorok kihagyása.

5.2. Dicsőséglista – `dicsoseglista.csv`

5.2.1. Mentés – `saveHighscore()`

- Append módban nyitja a filet
- Ha a file üres, először fejléct ír: „Név;Nyeremény;Dátum”
- Dátum: `localtime()`-mal kérdezi le, formátum: ÉÉÉÉ/HH/NN
- Sor formátuma: name;prize;date\n

5.2.2. Megjelenítés – `showHighscore()`

- Megnyitja olvasásra, átugorja a fejlécs sort
- Soronként: Név;Nyeremény;Dátum -> Dinamikus `HighScore` tömbbe, `strdup`-pal
- Rendezi csökkenő nyeremény szerint
- Kiíraskor a nyereményt `formatPrize()`-zal formázza, és igazított oszlopokban jeleníti meg.
- Végén felszabadít minden dinamikus stringet és az egész tömböt!!!!

6. Dinamikus memóriakezelés

6.1. Kérdések sztringjeinek tárolása

- A CSV beolvasás során minden sztringet `strdup()`-pal másol a saját memóriaterületére
 - Feleletválasztós: q, answers[0..3], correct, cat
 - Sorrendező: q, answers[0..3], correct, cat
- Felszabadításkor a megfelelő `free...Questions()` függvényt használjuk:
 - Végigmegy a kérdéseken, `free()` minden dinamikus mezőn.

6.2. Dicsőséglista dinamikus tömb

- `HighScore *entries = NULL;` – kezdetben üres
- Ha új elem kell, darabosan növeli a kapacitást (`capacity + 16`) meghívja a `realloc`-ot
- Minden sor, név és dátummezője `strdup`-olt string
- Hiba esetén (`NULL` `realloc` visszatérés) a már lefoglalt elemeket felszabadítja, hibaüzenetet ad
- *Kell egy temp pointer, mert nem tudnánk a már lefoglalt területeket `free()`-elni

6.3. Nyeremény formázása – `formatPrize()`

- BE: int (nyeremény Ft-ban)
- Először `snprintf()`-fel szöveggé alakítja pl. "5000000"
- Hátról haladva beszúr '.' karaktereket minden 3. karakter után -> "5.000.000"
- Egy dinamikus bufferre (out) ír, majd a végső eredményt `strdup()`-pal kimásolja, a buffert felszabadítja
- *A kódban használat után mindig fel kell szabadítani a lefoglalt helyeket

7. Segédfüggvények & Logika

7.1. Konzol

7.1.1. `consoleUtf8()`

- UTF-8 locale + Windows API beállítás.

7.1.2. `printSeparator()`

- Egységes elválasztó sort ír ki (pld: menük és kérdésszövegek közé)

7.1.3. `waitEnter()`

- „[Nyomj Entert a folytatáshoz...]” üzenetet ír ki
- Beolvas és eldob egy teljes sort (így addig leállítja a játékot)

7.1.4. `fileExists()`

- Létezésellenőrzés `fopen()`-nel (ha sikerül megnyitni, akkor létezik)

7.2. Biztonságos szintek

7.2.1. `isSafeLevel(int index)`

- Igazat ad vissza ha az index 4 vagy 9 (5. és 10. kérdések)

7.2.2. `getSafePrize(int index)`

- Kieséskor a legutóbb elért biztonságos szint pénzét adja vissza:
 - Ha `index >= 9`: 10. kérdés összege
 - Ha `index >= 4`: 5. kérdés összege
 - Ha egyik sem, 0 Ft

7.3. Segítségek

7.3.1. 50:50 – `print5050(ChooseQuestion *q, int hidden[2])`

- A helyes válasz indexét `q->correct[0]` - 'A'-val számolja ki (ASCII megoldás)
- Választ két véletlen rossz választ (`rand() % 4`), amely:
 - Nem eshet a helyes indexre
 - Nem ismétlődhet (`hidden[0] != hidden[1]`)
- Az eredmény: két index az `hidden` tömbben, melyeket a `playGame()` úgy használ, hogy ezek az opciók „kikapcsolódnak” és a játékos nem választhatja őket

7.3.2. Közönség segítség – `printAudience(ChooseQuestion *q)`

- A helyes válasz kap egy 30–60% közötti százalékot
- A maradék százalékot véletlenszerűen elosztjuk a többi három válasz között
- A végső eloszlást egy sorban írja ki: A: xx% B: yy% ...

8. Hibakezelés, Korlátok, Feltételezések

8.1. CSV hibák

- A sorok ellenőrzése pontosvesszők száma alapján. Ha nem egyezik: az adott sort kihagyjuk.
- Feltételezzük hogy a megadott csv fileok helyesek.

8.2. Maximális elemszámok

- A konstansok (pl. MAX_CHOOSE_QUESTIONS_PER_LEVEL, MAX_ORDER_QUESTIONS) korlátot vezetnek be a betölthető kérdésekre
- Ha egy szinten túl sok kérdés van, az azon felül levő sorok nem kerülnek beolvasásra

9. *Továbbfejlesztési ötletek

9.1. Dinamikus tömb használata a kérdésekre

9.2. Választható nehézségi szintek

9.3. Több játékmód

9.3.1. Igaz – Hamis

9.3.2. Időmérés (időre megy)

9.3.2.1. Sebesség / Pont alapján rendezni a dicsőséglistát